Healthcare Management System Automation Framework PROJECT DOCUMENTATION Java | Selenium WebDriver | Cucumber BDD | JUnit | Maven | Jenkins | GitHub 1. Project Overview The Healthcare Management System Automation Framework is an end-to-end UI test automation project for important workflows in a healthcare management application. It uses Java and Selenium WebDriver for browser automation, Cucumber BDD for readable scenarios, JUnit for test execution, and Maven for build and dependency management. Jenkins is integrated with the GitHub repository. A code change was pushed and a new Jenkins build was observed. The test target is a locally hosted healthcare demo application called CarePoint. Objectives - Automate healthcare application workflows. - Validate authentication and user access. - Test patient registration and appointment management. - Build reusable automation components. - Generate readable test reports. - Integrate test execution with CI. 2. Technology Stack Technology Purpose Java 17 Automation programming language Selenium WebDriver 4.25.0 Browser automation Cucumber 7.20.1 Behaviour-Driven Development JUnit 4.13.2 Test execution Maven 3.10.0 Build and dependency management Jenkins 2.589 Continuous Integration Git and GitHub Version control and source repository GitHub Webhooks Build triggering HTML Publisher Publishing HTML reports Runtime note: the automation project uses Java 17; Jenkins itself runs using Java 21. These are separate runtime configurations. 3. Project Architecture The framework follows a layered structure based on the Page Object Model (POM).Healthcare Management System Automation Framework Page 1

- Feature layer: human-readable Cucumber scenarios. - Step definition layer: connects Gherkin steps to Java methods. - Page object layer: page-specific locators and browser interactions. - Driver layer: manages Selenium WebDriver creation and configuration. - Hooks layer: setup and cleanup around scenarios. - Configuration layer: application and test configuration. - Reporting layer: Cucumber HTML, JSON and XML reports. - CI layer: Jenkins executes Maven tests and publishes reports. Execution flow GitHub push -> webhook/SCM trigger -> Jenkins -> Maven -> JUnit/Cucumber -> Selenium WebDriver -> CarePoint -> test results -> r 4. Project Directory Structure healthcare-automation-framework/ app/demo/index.html docs/TEST-PLAN.md scripts/run-local.sh scripts/run-tests.sh src/test/java/com/healthcare/ config/ConfigReader.java driver/DriverFactory.java hooks/Hooks.java pages/ AppointmentsPage.java BasePage.java DashboardPage.java LoginPage.java PatientsPage.java runner/RunCucumberTest.java steps/HealthcareSteps.java utils/WaitUtils.java src/test/resources/ config/config.properties features/ appointment-management.feature authentication.feature patient-management.feature session.feature testdata/users.properties .github/workflows/ui-tests.yml reports/ .gitignore pom.xml README.md 5. Functional Test Scenarios Scenario Purpose Admin login Verify administrator authentication and dashboard navigation. Staff login Verify staff authentication. Invalid login Verify incorrect credentials are rejected. Patient registration Verify patient registration workflow. Appointment booking Verify appointment booking workflow. Logout Verify an authenticated user can end a session. Observed result: six scenarios passed in successful test executions. This is a demonstration test suite, not evidence of production healthcare system validation.

6. Page Object Model The framework separates page-specific browser interactions from test scenarios. - LoginPage.java - login page actions and elements. - DashboardPage.java - dashboard interactions. - PatientsPage.java - patient management interactions. - AppointmentsPage.java - appointment management interactions. - BasePage.java - common page-level functionality. Benefits include reduced duplicated Selenium code, separation of locators from workflows, easier maintenance, improved readability and support for additional workflows. 7. Cucumber BDD Cucumber allows test scenarios to be written in Gherkin syntax. Java step definitions connect the readable steps to Selenium operations. Feature: Healthcare user authentication Scenario: Admin logs in successfully Given the user opens the healthcare application When the admin enters valid credentials And the admin clicks the login button Then the admin should see the dashboard Benefits include readable scenarios, separation of business behaviour from implementation, and improved test documentation. 8. Maven and Local Application Maven manages dependencies and test execution. The project uses Selenium Java 4.25.0, Cucumber 7.20.1, Cucumber JUnit 7.20.1 and JUnit 4.13.2. Run tests locally mvn clean test Start the local CarePoint demo application python -m http.server 8080 --directory app/demo Application URL: http://localhost:8080. Keep the local server running in a separate terminal while executing Selenium tests. 9. Jenkins CI Integration A Jenkins Freestyle job named Healthcare-Automation-Framework was configured. Jenkins is available locally at http://localhost:8081. - Configured Java 21 as the Jenkins runtime. - Configured Maven as a Jenkins build tool. - Configured a Windows batch build step to run mvn clean test. - Copied generated reports into the Jenkins workspace. - Configured HTML Publisher to publish the report. HTML Publisher configuration Setting Value HTML directory to archive reports Index page index.html Report title Healthcare Cucumber Test ReportHealthcare Management System Automation Framework Page 3

Troubleshooting achievement Cucumber reports were initially generated in the local project directory while HTML Publisher searched in the Jenkins workspace. This caused publishing to fail despite successful test execution. The build script was updated to copy HTML, JSON and XML reports into %WORKSPACE%\reports before publishing. This resolved the directory mismatch. 10. Cucumber Reporting The Cucumber runner generates console output and HTML, JSON and JUnit XML reports. plugin = { "pretty", "html:reports/cucumber.html", "json:reports/cucumber.json", "junit:reports/cucumber.xml" } - HTML: human-readable scenario and feature results. - JSON: structured results for downstream reporting integrations. - XML: JUnit-compatible results for CI integrations. The HTML report is copied to index.html in the Jenkins workspace for publication. 11. GitHub and Automatic Build Triggering The project was initialized as a Git repository, committed and pushed to GitHub: https://github.com/hrts51/healthcare-automation-framework git init git add . git commit -m "Initial commit - Healthcare Automation Framework" git branch -M main git remote add origin https://github.com/hrts51/healthcare-automation-framework.git git push -u origin main Jenkins was configured with the repository URL, branch specifier */main, and GitHub hook trigger for GITScm polling. A README change was committed and pushed; a new Jenkins build appeared without manually selecting Build Now. Verification caveat: the GitHub webhook page previously displayed a connection warning for a placeholder public URL. The observed automatic build confirms that a trigger path worked, but the exact webhook delivery configuration should still be checked before claiming every delivery is successful. 12. What Makes This Implementation Stand Out? Project-specific differentiators - Healthcare-oriented workflows rather than only a generic login demonstration. - A locally hosted CarePoint demo application used as a controllable test target. - Separate Java runtimes: Java 17 for the test project and Java 21 for Jenkins. - A practical Jenkins workspace/report-directory mismatch diagnosed and fixed by copying report files before publication. - An integrated GitHub-to-Jenkins workflow where a repository change was followed by a new Jenkins build. - Multiple report formats-HTML, JSON and XML-from the same Cucumber execution. - Separate local ports: CarePoint on 8080 and Jenkins on 8081. These are implementation differentiators, not claims that the underlying technologies or techniques are globally unique. The value is in combining and troubleshooting them into a working project. 13. Known Limitations and Accuracy Notes - The target is a local demo application, not a real healthcare provider system. - The current suite contains six scenarios and should be expanded. - The local application server must be running for UI tests. - The Jenkins job currently depends on a local project directory. - The webhook delivery configuration should be verified.

- The .github/workflows/ui-tests.yml file exists, but a successful GitHub Actions execution has not been established. - No production patient data or real medical records are involved. - Production security, performance, accessibility and API testing have not been established as implemented. 14. Recommended Future Improvements - Move Jenkins from the local Downloads path to a Pipeline job that checks out the repository. - Make Jenkins start and stop the CarePoint test application automatically. - Capture screenshots on test failure. - Expand negative, boundary and role-based scenarios. - Add patient search, update and appointment cancellation tests. - Add test data management and cleanup. - Add execution history and trend reporting. - Verify and document webhook deliveries. - Validate GitHub Actions separately if it is to be included as a project feature. - Add API testing and database validation when a suitable test environment is available. - Improve logging and failure diagnostics. 15. Resume-Ready Project Summary Healthcare Management System - Test Automation Framework Java | Selenium WebDriver | Cucumber BDD | JUnit | Maven | Jenkins | GitHub - Developed a healthcare UI automation framework using Java, Selenium WebDriver and Cucumber BDD, covering six scenarios across authentication, patient registration, appointment booking and logout. - Implemented Page Object Model with reusable page, driver, configuration and test utility components. - Integrated Jenkins with GitHub to automate Maven test execution following repository changes. - Configured Cucumber HTML, JSON and XML reporting and resolved Jenkins workspace report-publishing issues through automated report copying. 16. Interview Explanation "I developed a Healthcare Management System Automation Framework using Java, Selenium WebDriver, Cucumber BDD, JUnit and Maven. I automated important workflows such as admin and staff login, invalid login validation, patient registration, appointment booking and logout. I followed the Page Object Model to make the framework reusable and maintainable. I then integrated the project with Jenkins and GitHub. When code is pushed to GitHub, Jenkins can trigger a new build and execute the Maven test suite. One practical challenge I solved was Jenkins report publishing. My Cucumber reports were generated in the project directory, but Jenkins HTML Publisher expected them in its workspace. I updated the build process to copy the report files into the workspace before publishing them. This project helped me gain practical experience in UI automation, BDD, version control, CI integration, test reporting and troubleshooting."Healthcare Management System Automation Framework Page 5